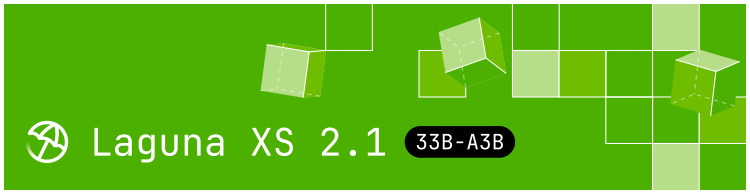
poolside/Laguna-XS-2.1

Text Generation · Transformers · Safetensors · laguna · laguna-xs-2.1 · vllm · conversational · custom_code · Eval Results · License: openmdw-1.1



Use on OpenRouter · Release blog post

Laguna XS 2.1

Laguna XS 2.1 is a 33B total parameter Mixture-of-Experts model with 3B activated parameters per token designed for agentic coding and long-horizon work on a local machine. This model is an upgraded version of our Laguna XS.2 model with a +5.4% jump on SWE-bench Multilingual as well as stronger performance on terminal-style tasks.

> For more details on how we train, including on data automixing and async off-policy agent RL, check out our recent technical report.

Highlights

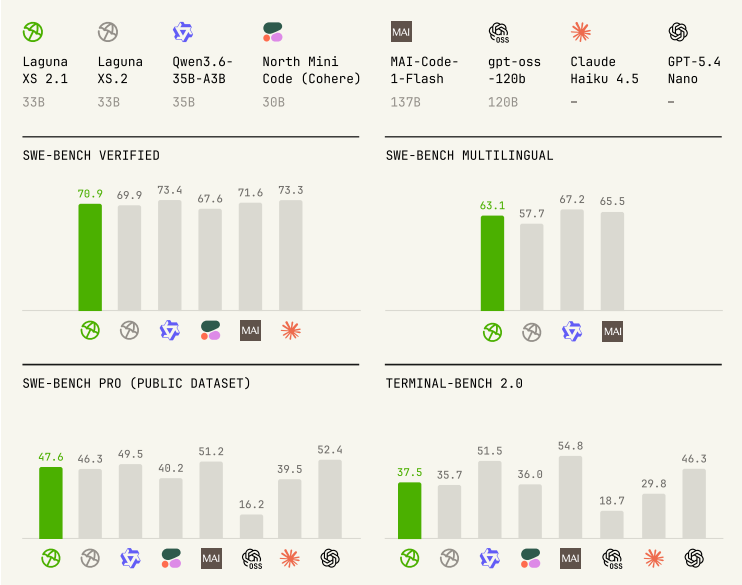
- **Mixed SWA and global attention layout**: Laguna XS 2.1 uses sigmoid gating with per-layer rotary scales, enabling mixed SWA (Sliding Window Attention) and global attention layers in a 3:1 ratio (across 40 total layers)
- **KV cache in FP8**: KV cache quantized to FP8, reducing memory per token
- **Native reasoning support**: Interleaved thinking between tool calls with support for enabling and disabling thinking per-request
- **Local-ready**: At 33B total parameters and 3B activated, Laguna XS 2.1 is compact enough to run on a Mac with 36 GB of RAM. Available on Ollama and llama.cpp. High-quality FP8, NVFP4 and INT4 quantized variants available (see the collection)
- **OpenMDW-1.1 license**: Use and modify the model and associated materials freely for commercial and non-commercial purposes (learn more about OpenMDW)

Model overview

- Training: pre-training, post-training and reinforcement learning stages
- Number of parameters: 33B total with 3B activated per token
- Optimizer: Muon
- Layers: 40 layers (10 layers with global attention, 30 layers with sliding window attention)

- Experts: 256 experts with 1 shared expert
- Sliding Window: 512 tokens
- Modality: text-to-text
- Context window: 262,144 tokens
- Reasoning support: interleaved thinking with preserved thinking

Benchmark results



Model	Size (total params.)	SWE-bench Verified	SWE-bench Multilingual	SWE-Bench Pro (Public Dataset
Laguna XS 2.1	33B	70.9%	63.1%	47.6%
Laguna XS.2	33B	69.9%	57.7%	46.3%
Qwen3.6-35B-A3B	35B	73.4%	67.2%	49.5%

Model	Size (total params.)	SWE-bench Verified	SWE-bench Multilingual	SWE-Bench Pro (Public Dataset
North Mini Code	30B	67.6%	-	40.2%
MAI-Code-1-Flash	137B	71.6%	65.5%	51.2%
gpt-oss-120B	120B	-	-	16.2%
Claude Haiku 4.5	-	73.3%	-	39.5%
GPT-5.4 Nano	-	-	-	52.4%

We used the highest publicly-referenced scores for all comparison models across each benchmark. In all cases these were official scores published in release blog posts or equivalent, with the exception of gpt-oss-120b and Claude Haiku 4.5 where the highest published (verified) scores for SWE-Bench Pro and Terminal-Bench 2.0 are from their respective official leaderboards.

▶ Expand for benchmarking methodology

Usage

Laguna XS 2.1 has launch-day support in vLLM, SGLang, Transformers and Llama.cpp, and TRT-LLM thanks to the support of the team at NVIDIA.

The fastest way to get started is using OpenRouter.

> We are providing free inference for a limited time for Laguna XS.2.1, as well as our larger 225B model, Laguna M.1. Visit our provider page on OpenRouter to get started.

pool

pool is a lightweight terminal-based coding agent and a dual Agent Client Protocol client-server.

Download and install for macOS and Linux:

```
curl -fsSL https://downloads.poolside.ai/pool
```

Launch and `> Log in with Poolside` to get a free, limited-use API key.

Alternatively, use Poolside models alongside the wide catalog of models available on OpenRouter by selecting `> Log in with OpenRouter`.

```
pool login
```

Use in any ACP client. Configure Zed and JetBrains automatically:

```
pool acp setup --editor zed|jetbrains
```

Use pool with Ollama with one-command setup:

```
ollama pull laguna-xs-2.1
ollama launch pool --model laguna-xs-2.1
```

Feedback and issues

Submit feedback with `/feedback` and read the full documentation on GitHub.

Local deployment

Laguna XS 2.1 is supported in vLLM, SGLang, Transformers and Llama.cpp, and TRT-LLM thanks to the support of the team at NVIDIA. Use Laguna-XS 2.1 with Ollama (with MLX support) and the mlx-lm framework for the best experience on your local machine.

vLLM

Serve Laguna XS 2.1 locally with vLLM and query it from any OpenAI-compatible client (see Controlling reasoning for tool calls, streaming, and reasoning extraction):

> Laguna XS 2.1 support is available in vLLM 0.21.0 and later (vllm-project/vllm#41129).

> Use a vLLM build that includes vllm-project/vllm#47311. Without that fix, the `poolside_v1` tool parser silently drops Laguna XS 2.1 tool calls that have no newline after the function name, and the raw tool-call markup leaks into the response. On an older build, pass `--tool-call-parser glm47` (the GLM 4.7 parser) instead.

```
pip install 'vllm>=0.21.0'

vllm serve \
    --model poolside/Laguna-XS-2.1 \
    --tool-call-parser poolside_v1 \
```

```
    --reasoning-parser poolside_v1 \
    --enable-auto-tool-choice \
    --served-model-name laguna \
    --default-chat-template-kwargs '{"enable
```

See the vLLM recipes page for additional deployment guidance.

> **Optional: speculative decoding with DFlash.** For lower latency, pair Laguna XS 2.1 with the DFlash speculator, a 5-layer Llama-style draft model that proposes up to 7 tokens per step at ~70% per-position acceptance on coding tasks. vLLM support is in progress in vllm-project/vllm#46853; once it lands, add `--speculative-config '{"model":"poolside/Laguna-XS-2.1-DFlash","num_speculative_tokens":7,"method":"dflash"}'` to the serve command above.

SGLang

Laguna XS 2.1 is supported in SGLang via sgl-project/sglang#24204. See the SGLang cookbook entry for a serving recipe.

```
git clone https://github.com/sgl-project/sgl
cd sglang
pip install -e "python[all]"

python -m sglang.launch_server \
  --model-path poolside/Laguna-XS-2.1 \
  --tp-size 8 \
  --mem-fraction-static 0.7 \
  --reasoning-parser poolside_v1 \
  --trust-remote-code
```

> **Optional: speculative decoding with DFlash.** The DFlash speculator can be paired with Laguna XS 2.1

> for lower latency. SGLang support was added in sgl-project/sglang#29446. Add `--speculative-algorithm DFLASH \ --speculative-draft-model-path poolside/Laguna-XS-2.1-DFlash-FP8` to the serve command above.

Transformers

Laguna XS 2.1 is supported in Transformers `v5.7.0` and later (huggingface/transformers#45673).

```
import torch
from transformers import AutoModelForCausalLI

model_id = "poolside/Laguna-XS-2.1"

tokenizer = AutoTokenizer.from_pretrained(mo
model = AutoModelForCausalLM.from_pretrained
    model_id,
    dtype=torch.bfloat16,
    device_map="auto",
)

messages = [
    {"role": "user", "content": "Write a Pyt
]

# Reasoning is on by default; pass enable_th
inputs = tokenizer.apply_chat_template(
    messages,
    add_generation_prompt=True,
    return_tensors="pt",
    enable_thinking=True,
).to(model.device)

outputs = model.generate(
    inputs,
    max_new_tokens=1024,
    do_sample=True,
    temperature=1.0,
    top_k=20,
```

```
)

response = tokenizer.decode(outputs[0][input
print(response)
```

TRT-LLM

Laguna XS 2.1 support is merged into TensorRT-LLM (NVIDIA/TensorRT-LLM#13559) and ships in the `v1.3.0rc16` pre-release wheels and later.

> A stable `v1.3.0` has not been released yet, so install a pre-release wheel from NVIDIA's package index (the latest stable, `v1.2.x`, does not include Laguna XS 2.1 support). To build from source instead, see the TensorRT-LLM build documentation. Laguna XS 2.1 support is on `main`.

Install the **CUDA-13** `torch` **build first**, then TensorRT-LLM. The default PyPI `torch` is a CUDA-12 build whose `cuda-bindings` pin conflicts with TRT-LLM's `cuda-python 13.x`, so a bare `pip install tensorrt-llm` fails to resolve; installing the cu130 `torch` up front avoids it.

```
# 1. CUDA-13 torch build (pins cuda-bindings
pip install 'torch==2.10.0' torchvision --in

# 2. TRT-LLM from NVIDIA's index (torch alre
pip install --pre 'tensorrt-llm>=1.3.0rc16'
    --extra-index-url https://pypi.nvidia.co
    --extra-index-url https://download.pytor
```

This resolves to `tensorrt-llm 1.3.0rc20` with `torch 2.10.0+cu130`, `cuda-python 13.0.3`, and `transformers 5.5.4`.

Load the checkpoint directly with `trust_remote_code=True`. No `transformers` compatibility overlay is required: `v1.3.0rc16+` pins `transformers 5.5.4`, which provides the symbols Laguna XS 2.1's config needs (earlier TRT-LLM releases pinned `transformers 4.57`, which did not).

```
from tensorrt_llm import LLM, SamplingParams

llm = LLM(
    model="poolside/Laguna-XS-2.1",
    trust_remote_code=True,
    tensor_parallel_size=1,
)

sampling = SamplingParams(max_tokens=1024, t
out = llm.generate(["Write a Python retry wr
print(out[0].outputs[0].text)
```

Or serve with an OpenAI-compatible endpoint:

```
trtllm-serve poolside/Laguna-XS-2.1 --port 8
```

The Laguna XS 2.1 tool-call and reasoning parsers are built into TRT-LLM `>=1.3.0rc16` (shipped with #13559), so no extra install is needed. Note that the flag names differ from vLLM's (`--tool_parser`, and the reasoning parser is `laguna`, not `poolside_v1`).

The same recipe works for the FP8 and NVFP4 variants: quantization is detected automatically from `quantization_config`, no extra flags required.

> **Optional: speculative decoding with DFlash.** The DFlash speculator can be paired with Laguna XS 2.1

> for lower latency. TRT-LLM support is in progress in NVIDIA/TensorRT-LLM#15666.

llama.cpp

> Requires building llama.cpp from the upstream PR that adds Laguna XS.2.1 support until it lands (ggml-org/llama.cpp#25165).

Official GGUF conversions (BF16 and Q4_K_M) are available at poolside/Laguna-XS-2.1-GGUF.

```
# Build llama.cpp from the PR branch
git clone https://github.com/ggml-org/llama.
git fetch origin pull/25165/head:laguna && g
cmake -B build && cmake --build build -j

# Download a GGUF and serve an OpenAI-compat
huggingface-cli download poolside/Laguna-XS-
./build/bin/llama-server -m ~/models/Laguna-
```

Ollama

Available on the Ollama library.

```
ollama run laguna-xs-2.1          # default
ollama run laguna-xs-2.1:q8_0     # higher p
ollama run laguna-xs-2.1:bf16     # full pre
```

Reasoning and tool-calling work out of the box via the built-in `laguna` template.

> **macOS (Metal) users:** `ollama run` may currently return empty output on Apple Silicon (a Metal f16 overflow in the MoE down-projection; fix pending in ggml-org/llama.cpp#25389). Until it ships, use a

> Linux/CUDA host or the `/api/generate` endpoint with `"raw": true`.

Atomic Chat

Laguna XS 2.1 is also available in Atomic Chat, a desktop app for running local models with a simple chat UI. To try it, download Atomic Chat, open the app, and choose Laguna XS 2.1 from the recommended models.

Controlling reasoning

Laguna XS 2.1 has native reasoning support and is designed to work best with *preserved thinking*, where `reasoning` content from prior assistant messages is preserved in the message history. This model will generally reason before calling tools and between tool calls.

Reasoning may not be generated in follow-up steps if prior thinking blocks are dropped (i.e., thinking is not preserved) when messages are reconstructed over multiple steps.

▶ Expand for example

Disabling reasoning

You can disable thinking by setting `enable_thinking` to `False` in a request or by not providing `--default-chat-template-kwargs {"enable_thinking": True}` or equivalent when starting the server.

▶ Expand for example

For agentic coding use cases, we recommend enabling thinking and preserving reasoning in message history as outlined in the Controlling reasoning section.

License

This model is licensed under the OpenMDW-1.1 License.

Intended and Responsible Use

Laguna XS 2.1 is designed for software engineering and agentic coding use cases, and you are responsible for confirming that it is appropriate for your intended application. Laguna XS 2.1 is subject to the OpenMDW-1.1 License, and should be used consistently with Poolside's Acceptable Use Policy. We advise against circumventing Laguna XS 2.1 safety guardrails without implementing substantially equivalent mitigations appropriate for your use case.

Please report security vulnerabilities or safety concerns to security@poolside.ai.